

# Acceso a Datos \_ Sesión 22-09-2025

## Marco normativo y contexto

- El título oficial de Técnico Superior en Desarrollo de Aplicaciones Multiplataforma.
- Módulo “Acceso a datos” como parte del 2º curso.
- Duración del módulo / horas oficiales: aproximadamente 80 horas

## Objetivos del módulo

Al finalizar el módulo, el alumnado deberá ser capaz de:

1. Conocer y manejar distintos tipos de almacenamiento de datos: ficheros, bases de datos relacionales, orientadas a objetos, no relacionales (documentales).
2. Diseñar el esquema lógico y físico de bases de datos, aplicando normalización y garantizando integridad, consistencia, seguridad y accesibilidad de los datos.
3. Establecer conexiones desde aplicaciones a bases de datos, ejecutar consultas, inserciones, modificaciones y eliminaciones de datos.
4. Utilizar procedimientos almacenados, triggers, vistas, y otras herramientas propias de los SGBD.
5. Trabajar con herramientas ORM (mapeo objeto-relacional) para facilitar el acceso a datos desde lenguajes orientados a objetos.
6. Manejar formatos de intercambio de datos (por ejemplo JSON, XML), serialización/deserialización, transformación.
7. Gestionar transacciones, concurrencia, bloqueo, y entender el rendimiento del acceso a datos.
8. Asegurar la persistencia y protección de los datos (backup, recuperación, gestión de errores).
9. Integrar los componentes de acceso a datos en aplicaciones multiplataforma cumpliendo especificaciones de calidad, eficiencia y usabilidad.

# Contenidos temáticos sugeridos

| Unidad                                                                             | Contenidos                                                                                                                                                                                                                                                                                                                                                                                                 |
|------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Unidad 1:<br/>Introducción al<br/>acceso a datos</b>                            | <ul style="list-style-type: none"><li>• Conceptos básicos: datos, información, almacenamiento, persistencia.</li><li>• Tipos de almacenamiento: ficheros vs. bases de datos.</li><li>• Modelos de datos: relacional, orientado a objetos, documental.</li><li>• Ventajas e inconvenientes de cada modelo.</li><li>• Requisitos no funcionales: seguridad, integridad, concurrencia, rendimiento.</li></ul> |
| <b>Unidad 2: Ficheros y<br/>formatos de<br/>intercambio</b>                        | <ul style="list-style-type: none"><li>• Manejo de ficheros: tipos (texto, binario), operaciones de lectura/escritura.</li><li>• Acceso secuencial y aleatorio.</li><li>• Serialización y deserialización de objetos.</li><li>• Formatos comunes de intercambio: XML, JSON, CSV.</li><li>• Parsers, validación, binding.</li><li>• Manejo de errores/excepciones relacionados con ficheros.</li></ul>       |
| <b>Unidad 3: Bases de<br/>datos relacionales</b>                                   | <ul style="list-style-type: none"><li>• Fundamentos: tablas, claves primarias y foráneas, esquemas.</li><li>• Diseño lógico: normalización (1NF, 2NF, 3NF, etc.).</li><li>• Diseño físico: índices, particionamiento, vistas.</li><li>• Lenguaje SQL: DDL, DML, DCL, consultas; joins; subconsultas; agregados; funciones.</li></ul>                                                                       |
| <b>Unidad 4:<br/>Procedimientos<br/>almacenados,<br/>triggers e integridad</b>     | <ul style="list-style-type: none"><li>• Definición y uso de procedimientos almacenados.</li><li>• Triggers (disparadores).</li><li>• Vistas y sinónimos.</li><li>• Integridad referencial y otras restricciones.</li><li>• Transacciones: definición, propiedades (ACID), aislamiento, bloqueo.</li></ul>                                                                                                  |
| <b>Unidad 5: Bases de<br/>datos no relacionales<br/>(documentales u<br/>otros)</b> | <ul style="list-style-type: none"><li>• Características de bases de datos no relacionales/documentales.</li><li>• Modelos de documentos (ej. JSON, BSON).</li><li>• Operaciones CRUD en bases de datos documentales.</li><li>• Indexación, consultas, agregaciones en bases no relacionales.</li></ul>                                                                                                     |

**Unidad 6: ORM y acceso desde lenguajes de programación**

• Principio del mapeo objeto-relacional. • Herramientas ORM populares (por ejemplo Hibernate, Entity Framework, etc.). • Configuración y uso de ORM: entidades/clases persistentes, relaciones, lazy loading, transacciones. • Comparación entre ORM vs SQL puro en determinados escenarios.

**Unidad 7: Conectividad y componentes de acceso**

• Drivers y conectores (JDBC, ODBC, otros). • Pooling de conexiones. • Componentes que abstraen el acceso a datos. • Seguridad: autenticación, autorización, cifrado en tránsito. • Pruebas de acceso a datos (unitarias, integración).

**Unidad 8: Rendimiento, optimización y buenas prácticas**

• Optimización de consultas y uso de índices. • Análisis de planes de ejecución. • Tuning de bases de datos. • Cacheado. • Gestión de concurrencia y conflictos. • Escalabilidad: particionamiento, replicación, clustering.

**Unidad 9: Integración y aplicaciones multiplataforma**

• Cómo integrar el módulo de acceso a datos en aplicaciones multiplataforma (web, móvil, escritorio). • Acceso remoto a bases de datos: servicios web, APIs. • Formatos de respuesta, JSON / XML / servicios REST, GraphQL. • Consideraciones de seguridad, latencia, conexiones móviles. • Persistencia en dispositivos móviles y sincronización.

**Unidad 10: Proyecto práctico / casos de estudio**

• Análisis de requerimientos de acceso a datos. • Diseño y construcción de una base de datos. • Implementación del acceso desde una aplicación real o simulada. • Integración de ficheros / intercambio de datos. • Optimización, seguridad. • Documentación y pruebas.

# Tema 1: Introducción al Acceso a Datos

## 1. Conceptos básicos

### 1.1. Dato e información

- **Dato:** representación mínima de un hecho, sin contexto.  
Ejemplo: 25
- **Información:** dato interpretado en un contexto.  
Ejemplo: 25 años (edad de una persona).

### 1.2. Persistencia

- **Persistencia:** propiedad que permite que los datos sobrevivan al fin de un programa.
- **Memoria volátil:** RAM (se borra al apagar).
- **Memoria persistente:** disco, SSD, base de datos.

#### Ejemplo en Java:

```
// Guardar un dato en memoria RAM
int edad = 25;
```

```
// Guardar un dato en un fichero (persistente)
import java.io.*;
```

```
public class GuardarDato {
    public static void main(String[] args) throws IOException {
        FileWriter fw = new FileWriter("edad.txt");
        fw.write("25");
        fw.close();
    }
}
```

## 2. Tipos de almacenamiento

### 2.1. Ficheros

- **Texto plano:** CSV, TXT → legibles por humanos.
- **Binarios:** estructuras codificadas → más rápidos, menos legibles.
- **Acceso:**
  - **Secuencial:** leer línea por línea.
  - **Aleatorio:** saltar a una posición del archivo (clase `RandomAccessFile` en Java).

### 2.2. Bases de datos

- **Relacionales (SQL)**
  - Modelo tabular.
  - Lenguaje: SQL.
  - Ejemplo: PostgreSQL, MySQL.
- **No relacionales (NoSQL)**
  - Flexibles, sin esquema fijo.
  - Ejemplo: MongoDB (documental), Redis (clave-valor), Neo4j (grafos).
- **Orientadas a objetos (OODBMS)**
  - Guardan objetos completos.
  - Ejemplo: ObjectDB.

## 3. Modelos de datos

### 3.1. Relacional

```
CREATE TABLE Alumno (  
  id SERIAL PRIMARY KEY,  
  nombre VARCHAR(100) NOT NULL,  
  edad INT CHECK (edad >= 0)  
);
```

```
INSERT INTO Alumno (nombre, edad) VALUES ('Laura', 22);  
SELECT * FROM Alumno;
```

### 3.2. Documental (MongoDB)

```
{  
  "id": 1,  
  "nombre": "Laura",  
  "edad": 22,  
  "asignaturas": ["AD", "PSP", "PMDM"]  
}
```

## 4. Requisitos del acceso a datos

- **Integridad:** claves, restricciones, normalización.
- **Concurrencia:** bloqueo, transacciones ACID.
- **Seguridad:** roles, permisos, cifrado.
- **Rendimiento:** índices, optimización de consultas, cacheado.

Ejemplo SQL de permisos:

```
GRANT SELECT, INSERT ON Alumno TO profesor;  
REVOKE DELETE ON Alumno FROM profesor;
```

## 5. Arquitectura de acceso a datos

- **Patrón DAO (Data Access Object):** capa que encapsula el acceso a datos.
- **ORM (Object Relational Mapping):** Hibernate, JPA, Entity Framework.
- **Conexión con JDBC (Java Database Connectivity):**

```
import java.sql.*;

public class ConexionBD {
    public static void main(String[] args) throws Exception {
        Connection conn = DriverManager.getConnection(
            "jdbc:postgresql://localhost:5432/academia", "postgres", "1234");

        Statement stmt = conn.createStatement();
        ResultSet rs = stmt.executeQuery("SELECT * FROM Alumno");

        while (rs.next()) {
            System.out.println(rs.getInt("id") + " - " + rs.getString("nombre"));
        }

        conn.close();
    }
}
```

# Ejercicios prácticos Tema 1

## Ejercicio 1 – Persistencia en ficheros

1. Crea un programa en Java que guarde en un fichero `alumnos.txt` los nombres de tres alumnos.
2. Haz otro programa que lea el fichero y muestre los nombres en consola.

## Ejercicio 2 – SQL básico

1. Crea una tabla `Asignatura` con los campos: `id`, `nombre`, `horas`.
2. Inserta al menos 3 registros.
3. Haz una consulta que muestre todas las asignaturas con más de 3 horas.

## Ejercicio 3 – Modelo documental

1. Representa el siguiente dato en formato JSON:
  - Un alumno con nombre *“Carlos”*, edad *21*, y asignaturas: *“AD”*, *“PSP”*.
2. Explica qué diferencia hay entre guardar este JSON en un fichero y almacenarlo en MongoDB.

## Ejercicio 4 – Conexión a base de datos

1. Conéctate desde Java a una base de datos PostgreSQL llamada `academia`.
2. Muestra en consola todos los alumnos de la tabla `Alumno`.